

CONSTRUCTION OS

PRD - Notifications & Event Centre

Full Product Requirements Document for Product, UX/UI, Frontend, Backend, Data, QA, Security, DevOps, SRE and Platform Operations Teams

Product purpose: Build the controlled event-processing, recipient-resolution and communication-delivery infrastructure that safely connects every ConstructionOS module without duplicating business state or bypassing permissions.

Document control	Value
Document	PRD_Notifications_Event_Centre
Version	1.0
Status	Implementation-ready functional and technical specification
Audience	Full-stack development, platform, security, QA, SRE and product teams
Primary routes	/notifications, /notification-preferences, /announcements, /admin/event-centre
Core architecture	Domain events + transactional outbox + policy engine + permission-safe recipient resolution + channel delivery
Access model	Deny-by-default, tenant-isolated, company/project scoped and sensitivity-aware
Workflow policy	Approval events supported; approval routing remains deferred to the final Approvals & Workflow PRD
Integration policy	In-app is mandatory core; email, push and SMS are optional adapters
Platform Configuration	Mandatory at module, event, policy, template, channel, audience, action and retention level
Date	2026-08-05

Table of Contents

1. Executive Summary
2. Product Vision, Outcomes and Failure Risks
3. Scope, Non-Goals and Assumptions
4. Architectural Principles
5. Domain Ownership and Source-of-Truth Boundaries
6. Personas, Roles and Access Model
7. Information Architecture
8. Four-Layer Reference Architecture
9. User Notification Centre
10. Global Bell and Notification Drawer
11. Categories, Severity, Priority and Urgency
12. Standard Event Envelope
13. Event Naming, Versioning and Schema Registry
14. Event Catalogue
15. Event Ingestion and Validation
16. Notification Policy Engine
17. Recipient Resolution
18. Permission-Safe Delivery and Redaction
19. Templates and Localization
20. Notification Lifecycle and User States
21. Delivery Channels and Routing
22. User Preferences and Precedence
23. Quiet Hours and Vacation Mode
24. Digests
25. Deduplication, Grouping and Replacement
26. Suppression, Expiry and Withdrawal
27. Reminder Engine
28. Escalation Engine
29. Acknowledgements
30. Announcements
31. Emergency Alerts
32. Subscriptions, Watching and Muting
33. Delivery Reliability
34. Transactional Outbox and Event Broker
35. Multi-Company, Group and Joint Venture Rules
36. Data Model
37. Backend Service Architecture
38. Domain Events and Integration Contracts
39. REST API Contract
40. Frontend Architecture and State
41. Search, Filters and Saved Views
42. Mobile, Responsive and Offline Behavior
43. Security, Privacy and Confidentiality
44. Platform Configuration
45. Administration and Operations Console
46. Observability and Audit
47. Analytics and Reporting
48. Performance, Scalability and Non-Functional Requirements
49. Validation, Error Handling and Concurrency
50. Retention, Deletion and Legal Hold
51. Testing Strategy and Critical Scenarios
52. Acceptance Criteria
53. Delivery Phases
54. Migration and Adoption
55. Page and Flow Registry
56. Risks and Open Decisions
57. Definition of Done

1. Executive Summary

The Notifications & Event Centre is the platform-wide communication delivery layer for ConstructionOS. It receives domain events from source modules, validates schemas, evaluates notification policies, resolves authorized recipients, composes permission-safe content, and delivers messages through the appropriate channel at the appropriate time.

The module is not a simple bell icon and must not become an alternative business workflow system. Source modules remain authoritative for tasks, contracts, deadlines, inspections, payments, permits, payroll and every other business state. The Event Centre owns communication intent, delivery state, read state, acknowledgement, retries, digests, escalation and observability.

Permanent rule: Source modules record what happened. The Event Centre interprets communication policy. The Notification Engine delivers the right information to the right authorized users at the right time.

1.1 Required outcomes

- No module may directly send uncontrolled user-facing notifications.
- Every processed event is idempotent, versioned, traceable and permission checked.
- Every notification links back to its authoritative source record.
- Critical alerts can bypass quiet hours only through configured mandatory policies.
- Users receive personal control over optional notifications without disabling mandatory safety, legal or security messages.
- External channels expose minimum safe information and never become the source of truth.
- Delivery failures, retries and dead-letter events are visible to platform operations.

1.2 Success indicators

Outcome	Target behavior
Correctness	Repeated broker delivery never creates duplicate user notifications.
Privacy	No recipient receives content for a record they cannot access at composition or delivery time.
Reliability	Temporary provider outages retry safely without losing or duplicating messages.
Relevance	Grouping, suppression and replacement prevent notification floods.
Control	Users can configure optional channels, quiet hours, digests and record subscriptions.
Governance	Mandatory announcements and emergency alerts have auditable acknowledgement.

2. Product Vision, Outcomes and Failure Risks

2.1 Product vision

Create one enterprise event and notification infrastructure that serves every ConstructionOS module, company, project and user while preserving source ownership, deny-by-default permissions and multi-company privacy.

2.2 Business outcomes

- Users receive timely, prioritized and contextual information instead of unstructured message noise.
- Executives, project teams and site personnel can rely on critical alerts without monitoring every module manually.
- Platform administrators can configure event policies without changing source-module code.
- Product teams can add new events through a governed Event Catalogue and schema contract.
- SRE teams can observe event latency, provider health, retry queues and delivery success.

2.3 Failure risks this PRD prevents

Failure mode	Consequence	Required control
Direct sending from source modules	Duplicate logic, inconsistent permissions and untraceable delivery	All user communication passes through the Event Centre.
No idempotency	Duplicate emails, push messages and inbox records	Event and delivery idempotency keys.
Permission checked only once	Sensitive data remains visible after access changes	Checks at recipient resolution, composition, delivery and open time.
Notification equals task	Users mistake information for accountable work	Tasks remain the execution system.
Unlimited broadcasting	Noise, privacy leakage and user distrust	Explicit recipient strategies and broadcast capability.
Provider dependency	Core operations fail when email/SMS provider is unavailable	In-app channel remains self-contained core.
No observability	Silent delivery failures	Processing monitor, delivery attempts and dead-letter queue.

3. Scope, Non-Goals and Assumptions

3.1 In scope

- Event ingestion, schema validation and Event Catalogue governance.
- Notification policies, recipient resolution and permission validation.
- In-app inbox, real-time updates, bell, drawer, search, grouping and archive.
- User preferences, quiet hours, digests, snooze, subscriptions and muting.
- Reminders, escalations, acknowledgements, announcements and emergency alerts.
- Optional email, browser push, native push and SMS adapter contracts.
- Delivery reliability, retries, rate limits, dead-letter handling and observability.
- Multi-company, project, group and Joint Venture scoping.
- Responsive mobile behavior and limited offline state synchronization.

3.2 Non-goals

- Business workflow routing, approval decisions, delegation or approval authority.
- Creating or changing the source business record from the notification service.
- Replacing Tasks, Calendar, activity timelines, chat or document required-reading features.
- Guaranteeing recall of content already delivered through external channels.
- Providing unrestricted public APIs for arbitrary message sending.
- Implementing every external provider in the first release.

3.3 Assumptions

- Every source module supports tenant-aware domain events and a transactional outbox.
- A central authorization interface can evaluate capabilities and record scope.
- ConstructionOS has a canonical User ID, Company ID, Project ID and deep-link format.
- All timestamps are stored in UTC and presented in user time zones.
- Platform Configuration can enable or disable events, channels and policies.

4. Architectural Principles

Principle	Requirement
Event driven	Business facts are published as versioned domain events.

Principle	Requirement
Transactional integrity	Source transaction and outbox event commit atomically.
Idempotency	Event processing, notification creation and provider delivery are retry safe.
Permission first	No content is composed or delivered before authorization succeeds.
Minimum disclosure	External channels use safe text and deep links to protected records.
Source ownership	Notifications never become the source of business truth.
Deterministic policy	Policies are versioned, auditable and reproducible.
User control	Optional events respect preferences, quiet hours, digests and mutes.
Mandatory safety	Configured emergency, legal and security alerts may override preferences.
Observable operations	Every stage exposes metrics, traces and failure states.

Implementation decision: Use domain events, transactional outboxes, idempotent consumers and permission checks at delivery time. Never allow modules to send uncontrolled user-facing messages.

5. Domain Ownership and Source-of-Truth Boundaries

Domain	Owner	Event Centre responsibility
Tasks	Tasks Module	Deliver assignment, mention, status and deadline notifications.
Calendar and Meetings	Calendar / Meetings	Deliver reminders, invitations and schedule-change notices.
Contracts	Contracts	Deliver obligation, expiry, amendment and risk alerts.
Finance	Finance	Deliver invoice, payment, budget and close-period notices.
Procurement	Procurement	Deliver RFQ, PO, delivery and supplier-performance alerts.
Inventory	Inventory	Deliver stock, count, expiry and movement alerts.
Assets	Assets	Deliver maintenance, inspection, reservation and compliance alerts.
Work Progress	Work Progress	Deliver delay, constraint, certification and progress events.
QA/QC and Handover	QA/QC	Deliver inspection, NCR, defect and readiness alerts.
HSE	HSE	Deliver incident, stop-work, permit and emergency alerts.
Government & Legal	Government & Legal	Deliver permit, obligation, hearing and legal restriction alerts.
HR and Payroll	HR / Payroll	Deliver contract, timesheet, payroll and certificate notices.
Approvals	Future Approvals Engine	Deliver request, return, decision and overdue events only.

5.1 Distinction from adjacent systems

- Notification is information; Task is accountable execution.
- Notification delivery history is not the source module audit trail.
- Calendar owns scheduled time; Notification owns delivery of reminders.
- Acknowledgement confirms receipt; it is not formal approval.
- Announcement broadcasts controlled information; it does not grant record access.

6. Personas, Roles and Access Model

Persona	Primary needs
General user	Relevant inbox, direct source links, read/archive/snooze, clear priorities.
Project or site manager	Project alerts, delays, safety and deadline visibility without notification overload.
Module owner	Governed event definitions and module-specific default policies.
Company administrator	Company announcements, mandatory policies and delivery reports.
Platform administrator	Global Event Catalogue, provider configuration and operational control.
HSE emergency role	Activate emergency alerts and monitor acknowledgement.
SRE / Platform Operations	Monitor broker lag, retries, providers and dead-letter events.
Security / Audit	Review recipient decisions, redaction and sensitive access history.

6.1 Suggested capabilities

```

notifications.view_self
notifications.manage_preferences
notifications.acknowledge
notifications.manage_subscriptions
announcements.create
announcements.publish
announcements.view_delivery
emergency_alerts.activate
event_catalogue.manage
notification_policies.manage
notification_templates.manage
delivery_failures.retry
dead_letter.replay
notification_audit.view

```

6.2 Access rules

- Users can access only their own Notification Centre unless a separate administrative capability applies.
- Administrative delivery history must not expose source content beyond the administrator's record access.
- Being a group, company, department or project administrator does not automatically create recipient access.
- Joint Venture records require explicit JV scope and cannot be inferred from parent-company membership.

7. Information Architecture

7.1 User-facing navigation

- All
- Important

- Mentions
- Assignments
- Deadlines
- Warnings
- Announcements
- Acknowledgement Required
- Snoozed
- Archived

7.2 Personal settings

- Notification preferences
- Channel preferences
- Digest preferences
- Quiet hours
- Project subscriptions
- Team subscriptions
- Watched records
- Muted records
- Language and time zone

7.3 Administration navigation

- Event Dashboard
- Event Catalogue
- Notification Policies
- Recipient Rules
- Templates
- Severity and Priority
- Grouping and Deduplication
- Reminder Rules
- Escalation Rules
- Digests
- Announcements
- Emergency Alerts
- Delivery Providers
- Delivery Failures
- Dead-Letter Queue
- Processing Monitor
- Analytics
- Settings

7.4 Primary routes

```
GET /notifications
GET /notification-preferences
GET /announcements
GET /admin/event-centre
GET /admin/event-centre/catalogue
GET /admin/event-centre/policies
GET /admin/event-centre/deliveries
GET /admin/event-centre/dead-letter
```

8. Four-Layer Reference Architecture

Layer	Responsibility	Core components
1. Event production	Source modules publish immutable business facts	Outbox, relay, schema version, correlation ID.

Layer	Responsibility	Core components
2. Policy and recipient resolution	Convert facts into communication intent	Catalogue, policy engine, recipient rules, permission adapter.
3. Delivery	Create inbox records and route channels	Composer, router, in-app service, provider adapters.
4. Governance and observability	Control, monitor and audit processing	Admin console, metrics, retries, dead-letter, analytics.

```

Source transaction + outbox
    |
    v
Event broker -> Ingestion -> Schema validation -> Policy evaluation
              -> Recipient resolution -> Permission checks -> Composition
              -> Scheduling/grouping -> In-app and optional external delivery
              -> Read/acknowledgement state + audit + metrics

```

8.1 Required service guarantees

- At-least-once event processing with idempotent effects.
- No user-visible notification before the source transaction commits.
- Ordered handling where aggregate order is business-critical.
- Backpressure, rate limiting and dead-letter handling under load.

9. User Notification Centre

9.1 Notification card

- Module icon
- Short title
- Permission-safe body
- Context label
- Company/project where permitted
- Timestamp
- Severity and priority
- Read state
- Acknowledgement state
- Primary deep link
- Overflow actions

9.2 User actions

- Mark read/unread
- Acknowledge
- Snooze
- Archive
- Mute event type
- Stop watching record
- Open source
- Report irrelevant notification

9.3 Bulk actions

- Mark selected as read
- Archive selected
- Snooze selected where compatible
- Clear expired informational messages
- Never bulk-acknowledge mandatory records without explicit confirmation

ID	Area	Requirement	Priority	Acceptance signal
NOT-001	Inbox	The user shall receive a persistent tenant-scoped inbox with cursor pagination.	Must	A user can load more records without offset drift.
NOT-002	Deep link	Every actionable notification shall open its authoritative source record.	Must	No copied business detail page exists in Notifications.
NOT-003	Read state	Read and unread state shall synchronize across active sessions.	Must	Bell count updates in real time.
NOT-004	Archive	Archiving shall remove a notification from active lists without deleting delivery history.	Must	Audit record remains queryable.
NOT-005	Redaction	A notification shall redact content when current source permission is lost.	Must	Deep link remains protected.

10. Global Bell and Notification Drawer

10.1 Bell behavior

- Unread count
- Critical indicator
- Latest notifications
- Mark all as read
- Open full centre
- Real-time update
- Keyboard and screen-reader access

10.2 Counter rules

- Exclude archived, expired and redacted informational messages.
- Count a grouped notification once rather than counting every child.
- Keep mandatory unacknowledged alerts visually distinct from unread count.
- Cap the displayed number while retaining the exact count in accessible text.

10.3 Drawer behavior

- Maximum recent items configurable
- Grouped cards expandable
- Context preview without loading forbidden fields
- Optimistic read/archive only
- No optimistic acknowledgement

11. Categories, Severity, Priority and Urgency

11.1 Default categories

- Assignment
- Mention
- Comment
- Deadline
- Reminder
- Status Change

- Document
- Finance
- Contract
- Procurement
- Inventory
- Asset
- Project
- Work Progress
- QA/QC
- HSE
- Government & Legal
- HR and Payroll
- Meeting
- System
- Announcement
- Emergency
- Custom

11.2 Independent dimensions

Dimension	Values	Purpose
Severity	Informational, Notice, Warning, High, Critical, Emergency	Describes business impact.
Priority	Low, Normal, Important, Urgent	Controls user ordering and visual emphasis.
Delivery urgency	Immediate, Standard, Delayed, Digest Only, Scheduled	Controls when delivery occurs.

11.3 Rules

- Color cannot be the only indicator.
- Severity does not automatically determine recipient access.
- Emergency bypass requires a configured mandatory policy.
- Priority may be user-sortable; severity is source/policy controlled.

12. Standard Event Envelope

```

event_id
event_name
event_version
occurred_at
published_at
tenant_id
group_id
company_id
project_id
actor_user_id
source_module
aggregate_type
aggregate_id
aggregate_version
correlation_id
causation_id
sensitivity_class
audience_context
deep_link
payload
idempotency_key

```

12.1 Mandatory validation

- Event ID and idempotency key are globally unique within the tenant.
- Event name and version exist in the Event Catalogue.
- Tenant and source aggregate identifiers are present.
- Occurred time cannot be after published time beyond configured clock tolerance.
- Payload conforms to registered JSON schema.
- Deep link follows the canonical internal route format.

12.2 Metadata rules

- Payload must not contain raw secrets.
- Sensitive values should be references rather than copied content.
- Actor may be system when no human initiated the event.
- Correlation and causation IDs are preserved end-to-end.

13. Event Naming, Versioning and Schema Registry

13.1 Naming convention

```
module.entity.event

tasks.task.assigned
tasks.task.overdue
documents.revision.uploaded
contracts.contract.expiring
inventory.stock.low
assets.maintenance.overdue
procurement.delivery.delayed
hse.incident.critical_reported
legal.permit.suspended
hr.payroll.payslip_published
```

13.2 Versioning rules

- Breaking payload changes require a new major event version.
- Non-breaking optional fields may be added within a supported version.
- Consumers declare supported versions.
- Unknown or invalid versions enter the dead-letter queue with a stable reason code.
- Historical events retain their original version for audit and replay.

13.3 Schema registry

- JSON Schema per event version
- Owner module
- Change history
- Compatibility status
- Sample payloads
- Deprecation date
- Consumer list
- Sensitivity classification

14. Event Catalogue

The Event Catalogue is the governed registry of every event allowed to create user-facing communication.

Field	Purpose
Event name and version	Stable technical identity.

Field	Purpose
Source module and owner	Accountability for payload and semantics.
Description	Human-readable business meaning.
Default category/severity/urgency	Baseline policy inputs.
Sensitivity class	Controls template variables and external delivery.
Recipient strategies	Allowed audience-resolution mechanisms.
Supported channels	Permitted delivery adapters.
Grouping and dedupe	Default grouping key and window.
Template	Default localization key.
Digest/ack/escalation capability	Feature eligibility.
Retention	Processing and user-record retention.
Enable state	Platform Configuration control.

14.1 Governance rules

- No arbitrary user-facing event text from source modules.
- Catalogue change is versioned and audited.
- Disabling an event suppresses future intents but does not delete history.
- Deprecated events remain processable during the supported transition period.

15. Event Ingestion and Validation

15.1 Processing stages

1. Consume broker message.
2. Validate transport metadata.
3. Check idempotency.
4. Resolve catalogue entry and schema.
5. Validate tenant and source context.
6. Persist EventProcessingRecord.
7. Evaluate notification policies.
8. Commit processing result and acknowledgement to broker.

15.2 Processing statuses

- Received
- Validated
- Policy Matched
- No Policy
- Suppressed
- Completed
- Failed Retryable
- Failed Permanent
- Dead Letter

15.3 Failure handling

Failure	Behavior
Invalid schema	Reject to dead letter; do not retry indefinitely.
Unknown catalogue event	Dead letter with EVENT_NOT_REGISTERED.
Temporary authorization outage	Retry with backoff; do not deliver.

Failure	Behavior
Policy service outage	Retry processing without duplicating prior effects.
Source record unavailable	Delay or suppress according to event policy.
Tenant mismatch	Permanent security failure and alert.

16. Notification Policy Engine

A Notification Policy converts one validated domain event into zero or more notification intents. Policies must be deterministic, versioned and reproducible.

16.1 Policy conditions

- Event type/version
- Payload values
- Company/project
- Business category
- Record status
- Financial threshold
- Risk/severity
- Deadline proximity
- Country/jurisdiction
- Working hours
- User preference
- Platform Configuration

16.2 Policy output

- Recipient strategy
- Template
- Severity
- Priority
- Delivery timing
- Allowed channels
- Acknowledgement requirement
- Grouping rule
- Escalation rule
- Expiry time
- Suppression rules

16.3 Policy precedence

9. Emergency mandatory policy
10. Legal/security mandatory policy
11. Company mandatory policy
12. Module default policy
13. User-specific optional preference
14. Record-level subscription or mute

16.4 Versioning

- Each notification stores policy ID and version.
- Queued intents retain the policy version that created them unless explicitly re-evaluated.
- Policy simulation must preview recipients and channels before activation.

17. Recipient Resolution

17.1 Supported strategies

- Explicit user IDs
- Responsible user
- Record creator
- Participants
- Watchers
- Mentioned users
- Project/site manager
- Contract owner
- Procurement buyer
- Asset custodian
- Assigned inspector
- Payroll employee
- Configured role within scope
- Subscription
- Announcement audience
- Escalation chain

17.2 Required validation

- User is active.
- User belongs to the tenant.
- Company/project scope matches.
- User currently has source-record access.
- Event and channel are enabled.
- User mute is respected unless policy is mandatory.
- Recipient has not already received the same intent.

17.3 Broadcasting restriction

No automatic department broadcasting: Department, company, team or project membership does not automatically create notification access. Recipient strategies must be explicit and permission validated.

ID	Area	Requirement	Priority	Acceptance signal
REC-001	Resolution	Recipient resolution shall produce a reason trace for every included and excluded user.	Must	Admin can inspect inclusion/exclusion reason without exposing source content.
REC-002	Scope	Role-based recipients shall be resolved inside the event company/project scope.	Must	No cross-company leakage.
REC-003	Inactive user	An inactive responsible user shall trigger configured fallback or suppression.	Must	No message is sent to disabled accounts.
REC-004	Deduplication	The same user resolved by multiple strategies shall receive one notification intent.	Must	Recipient merge is deterministic.

18. Permission-Safe Delivery and Redaction

18.1 Permission checkpoints

15. Recipient resolution
16. Sensitive template-variable hydration

- 17. External-channel composition
- 18. Final delivery dispatch
- 19. Notification open/deep-link resolution
- 20. Digest generation

18.2 Access revocation

- Deep links remain protected.
- Sensitive body text becomes redacted.
- Cached previews are invalidated.
- Search index entries are updated.
- Delivery history remains auditable.
- External content cannot be recalled, so it must be minimal from the start.

18.3 Safe external content

Unsafe	Safe alternative
Contract ABC worth EUR 4.2m failed legal review.	A restricted contract record requires your attention.
Employee salary changed to EUR 5,000.	A restricted HR record has been updated.
Fatal incident at Site B involving John Doe.	A critical HSE event requires authorized review.

18.4 Sensitivity classes

- Standard
- Internal
- Restricted
- Confidential
- Legal Privileged
- Payroll Restricted
- Medical Restricted
- Emergency

19. Templates and Localization

19.1 Template fields

- Short title
- Short body
- Expanded body
- Safe external body
- Module icon
- Context label
- Primary action label
- Deep link
- Localization key
- Fallback language
- Accessibility label

19.2 Registered variables

```

{{record_code}}
{{record_title}}
{{project_name}}
{{due_date}}
{{responsible_user}}
{{status}}
{{company_name}}

```

19.3 Localization rules

- User language
- Locale-aware dates
- User time zone
- Currency and unit formatting
- Pluralization
- Fallback chain
- Right-to-left readiness
- Template preview in every active language

19.4 Security rules

- Variables must be whitelisted per event and channel.
- HTML must be sanitized.
- Template rendering must fail closed when a required sensitive variable is unavailable.
- No executable code or arbitrary expressions inside templates.

20. Notification Lifecycle and User States

20.1 Processing lifecycle

```
Event Received -> Validated -> Policy Evaluated -> Recipients Resolved
-> Permission Checked -> Composed -> Scheduled/Queued -> Delivered
-> Read -> Acknowledged/Archived
```

20.2 Alternative processing states

- Suppressed
- Grouped
- Deferred
- Failed
- Retrying
- Dead Letter
- Expired
- Cancelled
- Permission Revoked

20.3 User-facing states

- Unread
- Read
- Snoozed
- Acknowledged
- Archived
- Expired
- Redacted
- Withdrawn

20.4 State rules

- Read and acknowledgement are independent.
- Archive never deletes delivery history.
- Withdrawn notifications preserve the withdrawal reason.
- Expired notifications may remain accessible in history according to retention policy.

21. Delivery Channels and Routing

Channel	Initial status	Requirements
In-app	Mandatory core	Persistent inbox, real-time updates, deep links, read/ack/grouping.
Browser push	Optional adapter	Permission state, service worker, safe text and click-through.
Email	Optional adapter	Verified address, safe subject/body, provider callbacks.
Native mobile push	Future/optional	Device tokens, badge sync, deep links and token revocation.
SMS	Critical optional	Verified phone, strict length/sensitivity controls and cost limits.

21.1 Routing inputs

- Severity
- Policy
- User preference
- Quiet hours
- Mandatory override
- Provider health
- Verified contact
- Sensitivity
- Delivery failure history

21.2 Fallback rules

- In-app is always created when the event is user-facing.
- External failure does not remove the in-app record.
- Fallback between external channels requires explicit policy.
- Emergency multi-channel delivery may run in parallel.

22. User Preferences and Precedence

22.1 Precedence order

```
Mandatory emergency policy
-> Mandatory company policy
-> Event administrator default
-> User event preference
-> User channel preference
-> Quiet hours
-> Digest preference
-> Record subscription or mute
```

22.2 User-configurable settings

- Enable/disable optional event type
- Preferred channels
- Immediate or digest
- Quiet hours
- Weekend rules
- Default snooze
- Digest time
- Project subscriptions
- Record watch state
- Language and time zone

22.3 Mandatory events

- Cannot be completely disabled.
- Must explain why the control is locked.
- May still minimize external detail.
- Must record override of quiet hours or user preferences.

23. Quiet Hours and Vacation Mode

23.1 Quiet-hour configuration

- Start/end time
- Time zone
- Working days
- Weekend behavior
- Holiday behavior
- Temporary pause
- Vacation mode

23.2 Behavior

- Critical/emergency may bypass.
- Standard external delivery is delayed.
- Digest-eligible notifications are collected.
- In-app records may still be created silently.
- Every bypass is auditable.

23.3 Time-zone safety

- Store schedule in user-local terms plus IANA time zone.
- Recalculate daylight-saving boundaries.
- Do not use fixed UTC offsets for recurring quiet hours.

24. Digests

24.1 Digest types

- Morning
- End-of-day
- Daily
- Weekly
- Project
- Manager
- Executive
- Custom

24.2 Generation rules

- Critical events are never held only for digest.
- Permissions are rechecked at generation time.
- Duplicates are collapsed.
- Resolved events may be excluded.
- Subject lines contain no restricted details.
- Links open filtered Notification Centre views.

24.3 Digest contents

- Action required
- Deadlines
- Mentions

- Assignments
- Warnings
- Grouped changes
- Source links
- Unresolved acknowledgement items

ID	Area	Requirement	Priority	Acceptance signal
DIG-001	Permission	Digest generation shall re-evaluate source access for each item.	Must	Removed access excludes or redacts the item.
DIG-002	Grouping	Digest items shall group by configured module/project/priority strategy.	Should	Users can scan the summary quickly.
DIG-003	Idempotency	A digest instance shall not be sent twice for the same period and recipient.	Must	Retry is safe.

25. Deduplication, Grouping and Replacement

25.1 Deduplication key

event_name + source_record + recipient + event_version + state + time_window

25.2 Grouping examples

- 8 new comments on Task T-104
- 5 documents updated in Project A
- 3 assets require inspection this week
- 12 inventory items are below minimum stock

25.3 Replacement examples

Task due tomorrow -> Task overdue
 Permit expiring -> Permit expired
 Delivery delayed -> Delivery received

25.4 Rules

- Grouping must retain child history.
- Replacement must preserve prior event references.
- Acknowledged mandatory records cannot be silently replaced.
- Grouped notifications use one unread count entry.

26. Suppression, Expiry and Withdrawal

26.1 Suppression reasons

- Actor equals recipient
- Newer event supersedes older event
- Source archived/deleted
- Deduplication window
- User mute
- Digest-only policy

- Permission removed
- State resolved before delivery
- Maintenance window

26.2 Expiry

- Every scheduled notification may define relevance expiry.
- Stale reminders must not deliver after expiry.
- Expired inbox records remain historically queryable according to retention.

26.3 Withdrawal

- Authorized source event may withdraw an announcement or alert.
- Withdrawal creates a visible state rather than deletion.
- External-channel recall is best effort only and must not be promised.

27. Reminder Engine

27.1 Sources

- Tasks
- Calendar
- Meetings
- Contracts
- Procurement
- Asset maintenance
- Inspections
- HR
- Government & Legal
- HSE
- QA/QC

27.2 Rule types

- Exact time
- Minutes/hours/days before
- Recurring until completed
- Recurring until acknowledged
- Business-day calculation
- Escalation after due date

27.3 Ownership

Boundary: Calendar or the operational module owns the source date and completion state. Notifications owns the reminder schedule and delivery state.

27.4 Rescheduling

- Source date changes invalidate future reminder instances.
- Already delivered reminders remain historical.
- Duplicate scheduler jobs are idempotent.

28. Escalation Engine

28.1 Escalation triggers

- Critical alert unacknowledged
- Corrective action overdue

- Repeated delivery failure
- Responsible user inactive
- Deadline passed
- Responsible role vacant
- Emergency acknowledgement missing

28.2 Targets

- Secondary responsible user
- Project manager
- Site manager
- Company administrator
- HSE manager
- Legal owner
- Configured role

28.3 Constraints

- Escalation never grants access.
- Escalated recipient must pass source permission.
- Escalation reason and chain are auditable.
- Loops and repeated escalation to the same user are prevented.

29. Acknowledgements

29.1 Applicable events

- Emergency instruction
- Mandatory policy
- HSE alert
- Stop-work notice
- Critical legal restriction
- Payslip/document publication
- Required-reading announcement
- System security notice

29.2 Acknowledgement record

- User
- Notification
- Acknowledged at
- Device/session
- Confirmation statement
- Optional comment
- Event version
- Document revision where applicable

29.3 Rules

- Acknowledgement is not approval.
- No bulk acknowledgement without explicit confirmation.
- Updated mandatory content may require a new acknowledgement.
- Acknowledgement cannot be backdated by normal users.

30. Announcements

30.1 Types

- Group
- Company

- Project
- Department
- Maintenance notice
- Policy update
- Operational notice
- Emergency communication
- System release
- Custom

30.2 Audience targeting

- Group
- Company
- Branch
- Department
- Team
- Project
- Site
- Role
- Selected users
- External portal audience later

30.3 Publishing controls

- Publish and expiry dates
- Pin
- Priority
- Required acknowledgement
- Attachments
- Translations
- Reminder
- Audience preview
- Recipient count
- Delivery report

30.4 Broadcast safeguard

Separate capability: Announcement creation and publication require dedicated capabilities. Audience preview must show scope and recipient count before publishing.

31. Emergency Alerts

31.1 Capabilities

- Bypass quiet hours
- Remain pinned
- Require acknowledgement
- Repeat at configured intervals
- Use multiple channels
- Show emergency contacts
- Display site/project scope
- Update as emergency changes

31.2 Lifecycle

Draft -> Active -> Updated -> Resolved -> Archived

31.3 Safety rules

- Only configured emergency roles may activate.
- Activation requires scope and clear instruction.
- Resolved alerts remain visible in history.
- Repeat delivery stops after acknowledgement or resolution according to policy.
- No rich confidential detail in SMS/push.

32. Subscriptions, Watching and Muting

32.1 Watchable records

- Project
- Task
- Contract
- Asset
- Procurement Package
- Document
- Client
- Work Package
- Inspection
- Legal Case
- Employee record where permitted

32.2 Subscription modes

- All activity
- Major changes only
- Mentions only
- Deadlines only
- Documents only
- Status changes only
- Custom

32.3 Rules

- Following never grants access.
- Muted records remain subject to mandatory policies.
- Source permission is checked at every delivery.
- Subscription is disabled automatically when access is permanently removed.

33. Delivery Reliability

33.1 Reliability model

- At-least-once event processing
- Idempotent consumers
- Idempotent notification creation
- Provider idempotency where supported
- Exponential backoff
- Rate limiting
- Circuit breakers
- Dead-letter queues

33.2 Delivery attempt states

- Pending
- Sending
- Delivered
- Provider Accepted

- Failed Temporary
- Failed Permanent
- Retrying
- Cancelled
- Expired

33.3 Retry policy

Condition	Behavior
Network/provider timeout	Retry with exponential backoff and jitter.
Invalid recipient address/token	Mark permanent failure; request contact remediation.
Rate limit	Honor retry-after and provider limits.
Expired notification	Cancel pending delivery.
Permission revoked	Cancel or redact before dispatch.
Provider outage	Open circuit and preserve in-app delivery.

34. Transactional Outbox and Event Broker

34.1 Source-module pattern

21. Write business state.
22. Write domain event into local outbox.
23. Commit both in the same database transaction.
24. Relay publishes event to broker.
25. Mark outbox event published using retry-safe state.

34.2 Broker requirements

- Tenant-aware partition strategy
- Durable retention
- Consumer groups
- Replay support
- Dead-letter topic
- Message headers for correlation and trace context
- Backpressure monitoring

34.3 Ordering

- Preserve aggregate ordering where required.
- Do not assume global ordering.
- Use aggregate version to reject stale state events.

```
Source DB transaction
- Update aggregate
- Insert outbox row
Commit
-> Outbox relay
-> Event broker
-> Event Centre consumer
```

35. Multi-Company, Group and Joint Venture Rules

- Every event and notification stores tenant, group, company, project and source ownership context.
- Child-company events are not automatically visible to parent-group users.

- Group administrators receive only explicitly configured events and must pass source permission.
- Joint Venture events remain isolated to JV scope and authorized participants.
- Intercompany events create separately scoped recipient decisions.
- Company transfer of a user does not rewrite historical delivery ownership.

35.1 Cross-company announcement

- Requires group-level broadcast capability.
- Shows company audience preview.
- Creates one recipient record per user with company context.
- Does not reveal child-company data in delivery reports beyond authorized scope.

36. Data Model

36.1 Core entities

```

EventCatalogueEntry
EventSchemaVersion
EventEnvelope
EventProcessingRecord
NotificationPolicy
PolicyCondition
RecipientRule
NotificationTemplate
TemplateTranslation
Notification
NotificationRecipient
NotificationGroup
NotificationDelivery
DeliveryAttempt
DeliveryProvider
UserNotificationPreference
UserChannelPreference
RecordSubscription
MutedRecord
ReminderRule
ReminderInstance
DigestDefinition
DigestInstance
EscalationRule
EscalationInstance
Acknowledgement
Announcement
AnnouncementAudience
SuppressionRecord
DeadLetterEvent
EventProcessingCheckpoint
NotificationMetricSnapshot

```

36.2 Notification fields

```

id
tenant_id
recipient_user_id
source_module
source_type
source_id
event_id
event_name
policy_id
policy_version

```

```

category
severity
priority
title
body
safe_external_body
deep_link
group_key
status
read_at
acknowledged_at
snoozed_until
expires_at
created_at
record_version

```

36.3 Index requirements

- recipient_user_id + status + created_at
- tenant_id + event_id
- source_type + source_id
- group_key + recipient
- delivery status + next_retry_at
- dead-letter status + event_name
- announcement audience and publish date

37. Backend Service Architecture

Service	Responsibility
Event Ingestion Service	Consume broker events and establish processing record.
Schema Registry	Resolve and validate event schemas.
Event Catalogue Service	Govern event semantics and defaults.
Notification Policy Service	Evaluate deterministic communication rules.
Recipient Resolution Service	Resolve candidate users and reason trace.
Permission Adapter	Authorize recipient, composition, delivery and open.
Composition Service	Render localized safe content.
Inbox Service	Persist user notifications and state changes.
Channel Router	Choose allowed delivery channels.
Delivery Orchestrator	Queue, retry and cancel delivery attempts.
Reminder Scheduler	Create and invalidate reminder instances.
Digest Service	Aggregate permission-safe digest instances.
Escalation Service	Resolve escalation chain and triggers.
Announcement Service	Manage broadcasts and delivery reports.
Acknowledgement Service	Record required receipt confirmations.
Suppression/Dedupe Service	Prevent duplicate or irrelevant notifications.
Dead-Letter Service	Inspect, remediate and replay failed events.
Search Indexer	Index permission-safe inbox content.
Metrics Service	Expose product and technical metrics.

37.1 Transaction boundaries

- Notification record and initial in-app delivery are committed atomically where possible.
- External delivery attempts are asynchronous.
- State transitions use optimistic concurrency.
- Provider callbacks are idempotent.

38. Domain Events and Integration Contracts

38.1 Events consumed

- All registered source-module events that have active notification policies.

38.2 Events produced

```

notifications.notification.created
notifications.notification.read
notifications.notification.acknowledged
notifications.notification.archived
notifications.delivery.succeeded
notifications.delivery.failed
notifications.digest.sent
notifications.escalation.triggered
notifications.announcement.published
notifications.emergency_alert.activated
notifications.dead_letter.created

```

38.3 Contract requirements

- Transactional outbox
- Stable event names and versions
- Correlation and causation IDs
- Tenant/company/project context
- No PII beyond registered schema
- Source deep link
- Aggregate version

39. REST API Contract

39.1 User notifications

```

GET    /notifications
GET    /notifications/counts
GET    /notifications/{id}
POST   /notifications/{id}/read
POST   /notifications/{id}/unread
POST   /notifications/{id}/acknowledge
POST   /notifications/{id}/snooze
POST   /notifications/{id}/archive
POST   /notifications/bulk-read
POST   /notifications/bulk-archive

```

39.2 Preferences and subscriptions

```

GET    /notification-preferences

```

```

PATCH /notification-preferences
GET /notification-subscriptions
POST /notification-subscriptions
DELETE /notification-subscriptions/{id}
POST /notification-mutes
DELETE /notification-mutes/{id}

```

39.3 Announcements

```

GET /announcements
POST /announcements
GET /announcements/{id}
PATCH /announcements/{id}
POST /announcements/{id}/publish
POST /announcements/{id}/withdraw
GET /announcements/{id}/delivery-report

```

39.4 Administration

```

GET /event-catalogue
POST /notification-policies
PATCH /notification-policies/{id}
POST /notification-templates
GET /notification-deliveries
GET /notification-failures
POST /notification-failures/{id}/retry
GET /event-dead-letter
POST /event-dead-letter/{id}/replay

```

39.5 API standards

- REST/JSON with OpenAPI
- Cursor pagination
- Whitelist filtering/sorting
- Idempotency-Key for writes
- If-Match or expected_version
- RFC 7807 problem details
- Tenant/actor context
- Stable business error codes

40. Frontend Architecture and State

40.1 Core components

```

NotificationBell
NotificationCounter
NotificationDrawer
NotificationCentrePage
NotificationCard
NotificationGroupCard
NotificationFilters
NotificationPreviewPanel
NotificationBulkBar
AcknowledgementModal
SnoozeMenu
PreferencePage
EventPreferenceMatrix
ChannelPreferencePanel

```

```

QuietHoursEditor
DigestSettings
SubscriptionManager
AnnouncementsPage
AnnouncementEditor
AudienceBuilder
DeliveryReportPage
EventAdminDashboard
PolicyBuilder
TemplateEditor
DeliveryFailureTable
DeadLetterViewer

```

40.2 State principles

- Server-state cache
- Cursor pagination
- Real-time incremental updates
- Optimistic read/archive only
- No optimistic acknowledgement
- Permission-invalidated cache
- URL-based filters
- Persisted personal preferences
- Accessible live-region updates

40.3 Real-time transport

- WebSocket or Server-Sent Events
- Resume cursor after reconnect
- Tenant/user channel isolation
- Heartbeat and connection-health monitoring
- Fallback polling where real time is unavailable

41. Search, Filters and Saved Views

41.1 Filters

- Read/unread
- Category
- Module
- Project
- Company
- Priority
- Severity
- Date
- Acknowledgement required
- Assignment
- Mention
- Archived
- Snoozed

41.2 Search rules

- Index only permission-safe content.
- Reindex or redact when permission changes.
- Never index safe-external and restricted body together without field access.
- Support prefix and exact record-code search.

41.3 Saved views

- Private by default

- User-defined name
- Filter and sort configuration
- Optional default Notification Centre view
- No sharing in initial scope unless permission model is defined

42. Mobile, Responsive and Offline Behavior

42.1 Responsive behavior

Viewport	Behavior
Desktop	Filter sidebar, stream, preview panel and bulk actions.
Tablet	Collapsible filters and slide-over preview.
Mobile	Single-column stream, swipe actions, filter sheet and pinned emergency alert.

42.2 Offline-supported actions

- View cached notifications
- Mark read locally
- Archive locally
- Prepare acknowledgement
- Queue state changes with client operation ID

42.3 Server-confirmed actions

- Formal acknowledgement
- Announcement publishing
- Emergency activation/resolution
- Mandatory preference changes
- Administrative retry/replay

42.4 Conflict handling

- Server state is authoritative for withdrawal, redaction and acknowledgement.
- Duplicate offline operations are idempotent.
- Conflicts show pending, synchronized, failed or superseded state.

43. Security, Privacy and Confidentiality

43.1 Security requirements

- Tenant isolation
- Company/project scope
- Permission checks before delivery
- Sensitive-content redaction
- Encrypted transport
- Encrypted provider credentials
- Rate limits
- Anti-spam controls
- Secure callback signatures
- No confidential payloads in logs
- Full audit trail
- Legal-hold compatibility

43.2 Threat controls

Threat	Control
Broadcast abuse	Separate capability, recipient preview, rate limits and audit.
Deep-link guessing	Authorization on every source route.
Template injection	Whitelisted variables and sanitized rendering.
Provider credential theft	Secrets vault, rotation and least privilege.
Webhook spoofing	Signed callbacks, replay protection and timestamp validation.
Notification phishing	Canonical ConstructionOS link validation and no arbitrary URL variables.
Cross-tenant event	Tenant validation and permanent security failure.

43.3 Privacy

- Minimize copied PII.
- Use references to source records.
- Apply retention by event class.
- Support user access/export requests without exposing other users.
- Mask contact details in administrative views.

44. Platform Configuration

- Enabled modules
- Event Catalogue entries
- Categories
- Severity/priority levels
- Policies
- Recipient rules
- Templates
- Channels
- Mandatory events
- Quiet-hour bypass
- Grouping/deduplication
- Suppression
- Reminder schedules
- Escalations
- Digests
- Announcement types
- Emergency roles
- Acknowledgement requirements
- Retention
- Provider adapters
- Localization
- Custom fields

44.1 Configuration layers

Layer	Examples
Platform	Global event registration, providers and severity taxonomy.
Group	Group announcements and mandatory policies.
Company	Company quiet-hour bypass, templates and recipients.
Project	Project subscriptions and project-specific escalation roles.

Layer	Examples
User	Optional event/channel preferences and digests.

44.2 Safe change management

- Draft and active versions
- Simulation before activation
- Effective date
- Change audit
- Rollback
- Impact summary
- No retroactive deletion of historical decisions

45. Administration and Operations Console

45.1 Event Dashboard

- Events received
- Processing latency
- Policy matches
- Suppressed count
- Notifications created
- Delivery success/failure
- Provider health
- Dead-letter count
- Broker lag

45.2 Policy Builder

- Condition editor
- Recipient strategy
- Template preview
- Channel rules
- Acknowledgement/escalation
- Simulation with sample event
- Version and activation

45.3 Delivery operations

- Failure list
- Attempt timeline
- Provider response
- Retry action
- Cancel stale delivery
- Contact remediation
- Bulk retry with safeguards

45.4 Dead-letter viewer

- Event payload with sensitive masking
- Failure reason
- Schema/version
- Processing history
- Source link
- Remediation note
- Replay with current or original policy option

46. Observability and Audit

46.1 Audit events

- Event received
- Schema validated
- Policy selected
- Recipient included/excluded
- Permission result
- Notification created
- Grouped/suppressed
- Delivery attempted
- Provider response
- Read
- Acknowledged
- Snoozed
- Archived
- Escalated
- Replayed
- Withdrawn

46.2 Technical telemetry

- Distributed traces
- Structured logs
- Metrics
- Broker lag
- Queue depth
- Provider latency
- Retry rate
- Circuit state
- Real-time connection health

46.3 Logging rules

- Never log full confidential bodies.
- Log stable IDs and classifications.
- Use correlation IDs.
- Separate security audit from operational debug logs.
- Apply retention and access controls.

47. Analytics and Reporting

47.1 Technical metrics

- Events received
- Processing latency
- Notification creation latency
- Delivery success/failure
- Retry volume
- Dead-letter count
- Provider latency
- Duplicate suppression
- Permission rejection
- Real-time connection health

47.2 Product metrics

- Read rate
- Acknowledgement rate

- Digest usage
- Snooze usage
- Mute rate
- Average unread count
- Notifications per user
- Notifications by module
- Emergency acknowledgement time

47.3 Safeguard

No performance judgement: Notification read, mute, snooze or acknowledgement metrics must not automatically become employee-performance ratings. Analytics must link to source data and describe behavior factually.

48. Performance, Scalability and Non-Functional Requirements

Area	Requirement
Availability	In-app inbox and event ingestion target enterprise availability; provider outages degrade externally only.
Latency	Normal in-app notifications should appear within seconds of event publication under nominal load.
Scale	Support high-volume group announcements and bursty incident events without service collapse.
Pagination	Cursor-based, stable under concurrent writes.
Search	Near-real-time indexing with permission invalidation.
Accessibility	WCAG 2.2 AA target, keyboard operation and non-color indicators.
Localization	Time-zone, language, pluralization and RTL readiness.
Resilience	Retry, circuit breaker, backpressure and dead-letter handling.
Maintainability	Versioned schemas, policies, templates and provider adapters.
Security	Deny-by-default, tenant isolation and minimum disclosure.

48.1 Capacity targets to confirm

- Peak events per second
- Peak recipients per event
- Maximum announcement audience
- Inbox retention volume
- Provider throughput
- Real-time concurrent connections
- Digest generation window

49. Validation, Error Handling and Concurrency

49.1 Error format

REST endpoints must use RFC 7807 problem details with stable ConstructionOS business codes, trace ID and field errors where applicable.

49.2 Stable business errors

```

EVENT_NOT_REGISTERED
EVENT_SCHEMA_INVALID
EVENT_VERSION_UNSUPPORTED
RECIPIENT_NOT_AUTHORIZED
NOTIFICATION_ALREADY_ACKNOWLEDGED
NOTIFICATION_EXPIRED
ANNOUNCEMENT_AUDIENCE_EMPTY
MANDATORY_POLICY_CANNOT_BE_MUTED
DELIVERY_PROVIDER_UNAVAILABLE
DEAD_LETTER_NOT_REPLAYABLE
VERSION_CONFLICT

```

49.3 Concurrency

- Use optimistic versioning for notification state changes.
- Acknowledge is idempotent.
- Read/archive race resolves to latest valid server state.
- Announcement publish is single-transition with idempotency key.
- Dead-letter replay creates a new replay attempt and does not erase original failure.

50. Retention, Deletion and Legal Hold

50.1 Record classes

Record	Retention behavior
User notification	Retain according to category and company policy; archive is not delete.
Delivery attempt	Retain operationally with provider response masking.
Acknowledgement	Longer controlled retention where legally or operationally required.
Emergency alert	Retain with full lifecycle and acknowledgement history.
Announcement	Retain published revision and audience/delivery report.
Event processing record	Retain for replay/audit window.
Dead-letter event	Retain until resolved and policy retention expires.

50.2 Deletion rules

- User deletion requests do not remove source-module audit history.
- Notification content may be anonymized or redacted according to policy.
- Legal hold blocks deletion or retention disposal.
- Provider logs follow provider-specific retention and data-processing agreements.

51. Testing Strategy and Critical Scenarios

51.1 Required test groups

- Schema validation
- Version compatibility
- Outbox delivery
- Duplicate event handling
- Recipient resolution
- Permission revocation

- Redaction
- Quiet hours/DST
- Digest generation
- Grouping/deduplication
- Escalation
- Acknowledgement
- Provider failure
- Retry/backoff
- Dead-letter replay
- Multi-company/JV isolation
- Emergency bypass
- Audience targeting
- Real-time delivery
- Offline sync
- Responsive UX
- Accessibility
- High-volume load

51.2 Critical scenarios

26. Same event is delivered three times by the broker.
27. User loses project access before email delivery.
28. Task completes before an overdue notification is sent.
29. Emergency occurs during quiet hours.
30. Provider accepts then later rejects delivery.
31. Policy changes while events are queued.
32. Group administrator lacks child-company access.
33. Responsible user becomes inactive.
34. Digest contains a record whose permission was removed.
35. Announcement targets 100,000 users simultaneously.
36. Source record is archived after notification creation.
37. Dead-letter event is replayed after underlying state changed.

51.3 Security tests

- Cross-tenant injection
- Template variable escalation
- Arbitrary URL injection
- Forged provider callback
- Broadcast abuse
- Sensitive log leakage
- Redacted search result
- Privilege removal during active session

52. Acceptance Criteria

ID	Area	Requirement	Priority	Acceptance signal
AC-001	Events	Registered versioned events are ingested idempotently.	Must	Duplicate broker delivery produces one processing effect.
AC-002	Outbox	Source transaction and event publication cannot diverge.	Must	Outbox recovery test passes.
AC-003	Policies	Policies are versioned, simulated and auditable.	Must	Notification stores policy/version.
AC-004	Recipients	Recipients are explicit and permission validated.	Must	No membership-only broadcast.

ID	Area	Requirement	Priority	Acceptance signal
AC-005	Privacy	Sensitive external text is minimized and source access is rechecked.	Must	Revoked user sees redaction.
AC-006	Inbox	Persistent in-app inbox supports real-time updates and cursor pagination.	Must	Multi-session count stays consistent.
AC-007	Preferences	Optional events respect channel, quiet-hour and digest preferences.	Must	Mandatory policy precedence is clear.
AC-008	Grouping	Duplicate and related events group or replace according to policy.	Must	No flood from repeated updates.
AC-009	Reliability	Temporary delivery failures retry safely.	Must	No duplicate external delivery after retry.
AC-010	Dead letter	Permanent processing failures are inspectable and replayable.	Must	Replay preserves original failure record.
AC-011	Acknowledgement	Required acknowledgements are immutable and auditable.	Must	Read alone does not satisfy acknowledgement.
AC-012	Announcements	Audience preview and delivery report are available.	Must	Broadcast capability is separate.
AC-013	Emergency	Emergency alerts can bypass quiet hours with audit.	Must	Pinned state and acknowledgement work.
AC-014	Multi-company	Company, group and JV isolation tests pass.	Must	No cross-scope leakage.
AC-015	Operations	Processing latency, provider health, retries and dead letters are observable.	Must	SRE dashboard is functional.
AC-016	Accessibility	Notification workflows meet keyboard and non-color requirements.	Must	Accessibility test suite passes.

53. Delivery Phases

Phase 1 - Core event and inbox

- Transactional outbox standard
- Event broker integration
- Event Catalogue
- Ingestion and schema validation
- Policy engine
- Recipient resolution
- Permission checks
- In-app Notification Centre
- Bell/drawer
- Read/archive
- Real-time updates
- Deep links
- Audit and monitoring

Phase 2 - Personal control

- Preferences
- Quiet hours
- Subscriptions
- Muting
- Grouping
- Deduplication
- Snooze
- Reminders
- Digests

Phase 3 - Enterprise communication

- Announcements
- Mandatory acknowledgement
- Emergency alerts
- Escalations
- Advanced policies
- Delivery reports

Phase 4 - External channels

- Email adapter
- Browser push
- Native push
- SMS for configured critical use
- Provider monitoring/failover

Final integration

- Approvals & Workflow events
- Delegation-aware recipients
- Workflow escalation
- Approval-decision notifications

54. Migration and Adoption

54.1 Migration goals

- Replace module-specific direct notifications with governed events.
- Register existing event types in the Catalogue.
- Migrate user preferences where available.
- Preserve prior notification history where technically feasible.
- Introduce external channels only after in-app stability.

54.2 Rollout

38. Instrument source modules with outbox.
39. Run event ingestion in shadow mode.
40. Compare expected recipients without delivery.
41. Enable in-app for pilot companies/projects.
42. Monitor noise and suppression.
43. Expand modules and tenants.
44. Enable announcements/emergency.
45. Add external providers.

54.3 Adoption safeguards

- Default preferences favor relevance over maximum volume.
- Provide clear explanation for mandatory events.
- Allow users to report irrelevant notifications.
- Review mute and flood metrics by event policy.

55. Page and Flow Registry

ID	Page/Flow	Primary purpose
NOT-P01	Notification Centre	User inbox, search, filters and actions.
NOT-P02	Notification Preferences	Per-event and channel configuration.
NOT-P03	Subscription Manager	Watched projects/records and mute state.
NOT-P04	Announcements	Create and manage broadcasts.
NOT-P05	Announcement Delivery Report	Recipient, delivery and acknowledgement visibility.
NOT-P06	Event Dashboard	Processing and provider operational overview.
NOT-P07	Event Catalogue	Registered events and schemas.
NOT-P08	Policy Builder	Conditions, recipients, templates and routing.
NOT-P09	Template Editor	Localized safe content.
NOT-P10	Delivery Failures	Inspect and retry failed attempts.
NOT-P11	Dead-Letter Viewer	Remediate and replay failed events.
NOT-F01	Event to in-app	Consume, validate, resolve, compose and deliver.
NOT-F02	Emergency activation	Create, target, publish, repeat and resolve.
NOT-F03	Digest generation	Select, authorize, group and deliver digest.
NOT-F04	Acknowledgement	Open, confirm and record receipt.
NOT-F05	Access revocation	Redact cached content and protect deep link.

56. Risks and Open Decisions

56.1 Major risks

Risk	Mitigation
Notification overload	Conservative defaults, grouping, replacement, suppression and user feedback.
Permission leakage	Repeated authorization checks and minimal external content.
Event contract drift	Schema registry, versioning and consumer compatibility.
Provider lock-in	Adapter abstraction and in-app mandatory core.
Emergency abuse	Restricted capabilities, clear scope and full audit.
Replay changes business meaning	Replay preview and original/current policy selection.
High-volume broadcast load	Batch fan-out, queue partitioning and backpressure.

56.2 Open decisions before implementation lock

- Event broker technology and partition strategy
- Real-time transport choice
- Initial external provider(s)

- Default retention by category
- Maximum broadcast audience per operation
- Whether email is Phase 1 or Phase 4
- SLA and capacity targets
- Policy expression language
- Provider data residency requirements

57. Definition of Done

- All mandatory requirements and acceptance criteria pass automated and manual QA.
- Event schemas and Catalogue entries are documented and versioned.
- Source modules use transactional outbox for enabled events.
- Permission and multi-company isolation tests pass.
- In-app inbox, bell, drawer and preferences are responsive and accessible.
- Retries, circuit breakers and dead-letter replay are operationally verified.
- Observability dashboards and alerting are active.
- Security review confirms safe templates, provider callbacks and secret handling.
- Load tests cover peak events, announcements, digests and concurrent connections.
- Platform Configuration controls are functional without code changes.
- Runbooks exist for provider outage, broker lag, duplicate events and dead-letter growth.
- Product, Engineering, QA, Security and Platform Operations sign off the release.

Final architectural position: The module is complete only when every communication is traceable from source event through policy, recipient decision, permission check, delivery attempt and user state - without becoming a duplicate workflow or business-data system.